

UNIVERSITÀ
DEGLI STUDI
DI MILANO

Workshop: Programming Lab

HUMAN-CENTERED ARTIFICIAL INTELLIGENCE

Lecture 6: Tuples, Sets and Dictionaries

Dr. Fatme Hachem

09/11/2026

Lists and tuples

What is the difference between these 2 pieces of code:

```
a= [1,2,3]
print(a[0])
a[0]=-1
print(a)
```

```
a= (1,2,3)
print(a[0])
a[0]=-1
print(a)
```

Lists and tuples

What is the difference between these 2 pieces of code:

```
a= [1,2,3]
print(a[0])
a[0]=-1
print(a)
```

a is a **list**

```
1
[-1, 2, 3]
```

```
a= (1,2,3)
print(a[0])
a[0]=-1
print(a)
```

a is a **tuple**

```
1
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[142], line 3
      1 a= (1,2,3)
      2 print(a[0])
----> 3 a[0]=-1
      4 print(a)

TypeError: 'tuple' object does not support item assignment
```

Tuples

Tuples are like lists in Python, they are used to store collections of items.

But:

	List	Tuple
Mutability	Yes	No
Syntax	defined using Square brackets	defined using Parentheses

Tuples are faster and more memory-efficient than lists, especially for large datasets, because they are immutable. Once a tuple is created, it cannot be changed, which simplifies their internal representation in memory.

Sets

Sets in python are **unordered** collections of **unique** elements.

Sets:

- Do not allow duplicate elements.
- Unordered (Do not maintain any order of the elements)
- Mutable

Sets: creation

Set are created using curly braces **{ }**

```
set1= {1,2,3}
```

Or, by using the **set()** constructor.

The set() constructor takes a list of items as an argument and makes a new set that contains those items.

```
set2= set([1,2,3])
print(set2)
```

```
{1, 2, 3}
```

Sets: creation

What if ….

```
l=[1,2,3,4,3,2]

set3=set(l)
```

The list l contains duplicate elements. What will happen when we try to create a set from l?

```
print(set3)
```

```
{1, 2, 3, 4}
```

Sets: adding elements

Adding elements: add()

```
s= {1,2,3,4}
s.add(5)
print(s)
```

```
{1, 2, 3, 4, 5}
```

Sets: removing elements

Removing elements: remove() or discrad()

```
s.remove(3)
print(s)
```

```
{1, 2, 4, 5}
```

```
s.discard(4)
print(s)
```

```
{1, 2, 5}
```

Sets: removing elements

What's the difference between remove() and discard()?

With the remove() methods:

If the element is not present, it will raise an error.

With the discard() method:

If the element is not present, it does nothing and does not raise an error.

```
s.discard(10)
print(s)
```

```
{1, 2, 5}
```

```
s.remove(10)
print(s)
```

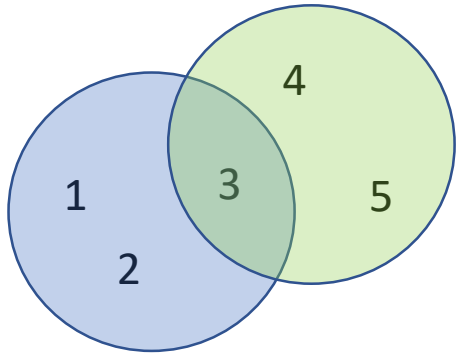
```
---------------------------------
KeyError
Cell In[152], line 1
----> 1 s.remove(10)
      2 print(s)

KeyError: 10
```

Sets operations: Union

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
union_set = set1.union(set2)
print(union_set)
```

```
{1, 2, 3, 4, 5}
```

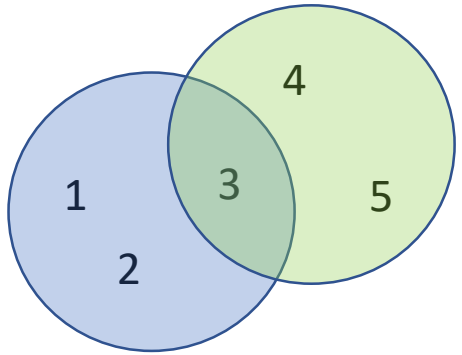


The operand + is unsupported for the sets

Sets operations: Intersection

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
intersection_set = set1.intersection(set2)
print(intersection_set)
```

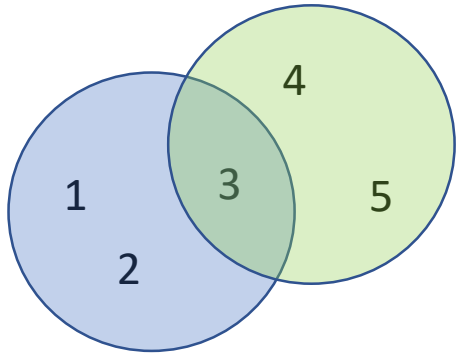
```
{3}
```



Sets operations: Difference

```
difference_set = set1.difference(set2)
print(difference_set)
```

```
{1, 2}
```




```
difference_set = set2.difference(set1)
print(difference_set)
```

```
{4, 5}
```

Dictionary

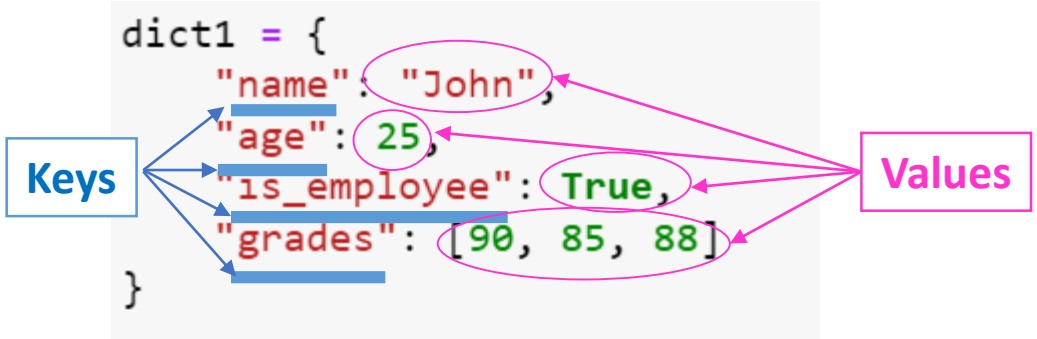
Dictionaries in Python are unordered collections of items, but those items are key-value pairs.

```
# Creating an empty dictionary
empty_dict = {}

# Creating a dictionary with

dict1 = {
    "name": "John",
    "age": 25,
    "is_employee": True,
    "grades": [90, 85, 88]
}

dict2 = {
    "name": "Bob",
    2: 23,
    3: True
}
```



- Keys are unique within a dictionary, while values may not be.
- The values of a dictionary can be of any data type, but the keys must be of an immutable data type such as strings, numbers, or tuples.
- Dictionaries are also known as associative arrays or hash maps in other programming languages.

Dictionaries: accessing and modifying values

We can access a value in a dictionary by its key:

```
dict1 = {
    "name": "John",
    "age": 25,
    "is_employee": True,
    "grades": [90, 85, 88]
}

print(dict1["name"])
print(dict1["age"])
```

Values can be modified, by specifying the corresponding keys as well:

```
dict1["age"]=26
dict1["grades"].append(79)

print(dict1)
```

```
{'name': 'John', 'age': 26, 'is_employee': True, 'grades': [90, 85, 88, 79]}
```

Dictionaries: useful methods

method	description
dict1.keys()	Return the dictionary's keys
dict1.values()	Return the dictionary’s values
dict1.items()	Return the dictionary's keys-value pairs
dict1.get(key, default)	Returns the value for the given key, or what is specified as default if the key is not found.
dict1.pop(key, default)	Removes the item with the given key, and returns its value or what is specified as default if the key is not found.
dict1.clear()	Removes all items from the dictionary
len(dict1)	Returns the number of items in the dictionary

Dictionaries: useful methods

```
dict1 = {
    "name": "John",
    "age": 25,
    "is_employee": True,
    "grades": [90, 85, 88]
}

print(dict1.keys())
print(dict1.values())
print(dict1.items())
```

```
dict_keys(['name', 'age', 'is_employee', 'grades'])
dict_values(['John', 25, True, [90, 85, 88]])
dict_items([('name', 'John'), ('age', 25), ('is_employee', True), ('grades', [90, 85, 88])])
```

The keys() method returns a **view object** that displays a list of all the keys in the dictionary in order of insertion.

This view object changes according to the changes in the dictionary.

Dictionaries: useful methods

```
d = {
    "name": "John",
    "age": 25,
    "is_employee": True,
    "grades": [90, 85, 88]
}

k=d.keys()
print("k is  ", k)

d["university"]='Economics'
print("after adding university, k is  ", k)
```

The keys() method returns a **view object** that displays a list of all the keys in the dictionary in order of insertion.

This view object changes according to the changes in the dictionary.

```
k is   dict_keys(['name', 'age', 'is_employee', 'grades'])
after adding university, k is   dict_keys(['name', 'age', 'is_employee', 'grades', 'university'])
```

Dictionaries: useful methods

```python
d = {
    "name": "John",
    "age": 25,
    "is_employee": True,
    "grades": [90, 85, 88]
}

k=d.keys()
print("k is  ", k)

d["university"]='Economics'
print("after adding university, k is  ", k)

l=list(k)
print("l is  ", l)

d["city"]='Milano'
print("after adding city, k is ", k)
print("after adding city, l is ", l)
```

If we want to use the keys as list, we need to convert the object view explicitly to a list type using the list() constructor.

Same for values() and items() methods

```
k is   dict_keys(['name', 'age', 'is_employee', 'grades'])
after adding university, k is   dict_keys(['name', 'age', 'is_employee', 'grades', 'university'])
l is   ['name', 'age', 'is_employee', 'grades', 'university']
after adding city, k is  dict_keys(['name', 'age', 'is_employee', 'grades', 'university', 'city'])
after adding city, l is  ['name', 'age', 'is_employee', 'grades', 'university']
```

Dictionaries: useful methods

```
dict1 = {
    "name": "John",
    "age": 25,
    "is_employee": True,
    "grades": [90, 85, 88]
}
```

```
print(dict1["name"])
```

```
John
```

```
print(dict1["namee"])
```

```
---------------------------------------
KeyError
Cell In[45], line 1
----> 1 print(dict1["namee"])

KeyError: 'namee'
```

```
print(dict1.get("name"))
```

```
John
```

```
print(dict1.get("nameee"))
```

```
None
```

```
print(dict1.get("nameee", -1))
```

```
-1
```

Exercise 1

Ask the user to enter two strings. Find and print the common letters (characters) in both strings using sets.

Exercise 1

Ask the user to enter two strings. Find and print the common letters (characters) in both strings using sets.

```
# Ask the user to enter two strings
string1 = input("Enter the first string: ")
string2 = input("Enter the second string: ")

set1= set(string1)
set2= set(string2)
common_letters = set1.intersection(set2)

print(common_letters)
```

Exercise 2

Ask the user to enter a sentence. Split the sentence into words and create a set of unique words. Count and print the number of unique words in the sentence.

Exercise 2

Ask the user to enter a sentence. Split the sentence into words and create a set of unique words. Count and print the number of unique words in the sentence.

```
# Ask the user to enter a sentence
sentence = input("Enter a sentence: ")

# Split the sentence into words and create a set of unique words
words = sentence.split()
unique_words = set(words)

# Count the number of unique words
number_of_unique_words = len(unique_words)

# Print the number of unique words
print("Number of unique words in the sentence:", number_of_unique_words)
```

```
Enter a sentence: Hello how are you
Number of unique words in the sentence: 4
```

Exercise 3

Given the tuple (1,2, ‘red’), write a python program to reverse this tuple.

Exercise 3

Given the tuple (1,2, ‘red’), write a python program to reverse this tuple.

```python
# Original tuple
original_tuple = (1, 2, 'red')

# Reversing the tuple
reversed_tuple = original_tuple[::-1]

# Print the reversed tuple
print("Reversed tuple:", reversed_tuple)
```

Exercise 4

Create a simple shopping cart system in Python.

The program should repeatedly ask the user to enter:

- the name of a product
- the price of that product

The user can finish entering products by typing “done” instead of a product name.

If the same product is entered more than once, do not add it as a new item. Instead:

- increase its quantity by 1
- update its unit price to the most recently entered price.

For example, if `apple` is entered three times, its quantity should be 3. If the prices entered were 1.00, 1.20, and 1.10, the final unit price of `apple` should be 1.10.

Once the user enters `done`, display a summary of the shopping cart. For each product, show:

product name, quantity, unit price and the total price for that product (quantity × unit price).

Finally, display the overall total cost of the shopping cart.

Exercise 4

What is the "item" we are collecting?

It is a product, represented by its name, quantity, and total price.

We can represent it as an element in a dictionary ***shopping_cart***:

- The key is the name of the product itself (e.g., 'apple', 'orange', 'bread').
- The value can be another dictionary, ***dict_product_detail***, that contains:
  - 'quantity' as a key, with the value initialized to 1, and incremented each time the user adds the same product.
  - 'price' as a key, with the value initialized to the price of the first occurrence of the product, and updated by adding the additional prices each time the user adds the same product later.

Exercise 4

```
Enter the product name (or type 'done' to finish):  bread
Enter the price for bread:  3
```

‘bread’: {‘quantity’: 1, ‘price’: 3}

shopping_cart

{}

Exercise 4

```
Enter the product name (or type 'done' to finish):  bread
Enter the price for bread:  3
```

‘bread’: {‘quantity’: 1, ‘price’: 3}

key value key value

dictionary ***dict_product_detail***
Which is also
the **value** of the key ‘bread’

shopping_cart

{}

Exercise 4

```
Enter the product name (or type 'done' to finish):  bread
Enter the price for bread:  3
```

‘bread’: {‘quantity’: 1, ‘price’: 3}

key value key value

dictionary ***dict_product_detail***
Which is also
the **value** of the key ‘bread’

shopping_cart
{
‘bread’: {‘quantity’: 1,
‘price’: 3}
}

Added as an element (pair key:value) to the dictionary shopping_cart

Exercise 4

```
Enter the product name (or type 'done' to finish):  orange
Enter the price for orange:  3
```

‘orange’: {‘quantity’: 1, ‘price’: 3}

key value key value

dictionary ***dict_product_detail***
Which is also
the **value** of the key ‘orange’

Added as an element (pair key:value) to the dictionary shopping_cart

shopping_cart

{

‘bread’: {‘quantity’: 1, ‘price’: 3},

‘orange’: {‘quantity’: 1, ‘price’: 3}

}

Exercise 4

```
Enter the product name (or type 'done' to finish):  orange
Enter the price for orange:  4
```

‘orange’ exists already in the dictionary **shopping_cart**

=> We upate the existing value of that key, instead of creating a new pair.

‘orange’: {‘quantity’: ~~1~~ 2, ‘price’: ~~3~~ 7}



updating the value of the pair where the key=‘orange’

shopping_cart

{

‘bread’: {‘quantity’: 1, ‘price’: 3},

‘orange’: {‘quantity’: 2, ‘price’: 7}

}

Exercise 4

```
Enter the product name (or type 'done' to finish):  bread
Enter the price for bread:  3
Enter the product name (or type 'done' to finish):  orange
Enter the price for orange:  3
Enter the product name (or type 'done' to finish):  apple
Enter the price for apple:  two
Invalid price, please enter a number.
Enter the price for apple:  2
Enter the product name (or type 'done' to finish):  orange
Enter the price for orange:  4
Enter the product name (or type 'done' to finish):  biscuit
Enter the price for biscuit:  1
Enter the product name (or type 'done' to finish):  done
Shopping Cart Summary:
Item:  bread , Quantity:  1 , Total Price:  3.0
Item:  orange , Quantity:  2 , Total Price:  7.0
Item:  apple , Quantity:  1 , Total Price:  2.0
Item:  biscuit , Quantity:  1 , Total Price:  1.0
Total Cost:  13.0
```

shopping_cart

{

‘bread’: {‘quantity’: 1, ‘price’: 3},

‘orange’: {‘quantity’: 2, ‘price’: 7},

‘apple’: {‘quantity’: 1, ‘price’: 2},

‘biscuit’: {‘quantity’: 1, ‘price’: 1}

}

Exercise 4

```
Enter the product name (or type 'done' to finish):  bread
Enter the price for bread:  3
Enter the product name (or type 'done' to finish):  orange
Enter the price for orange:  3
Enter the product name (or type 'done' to finish):  apple
Enter the price for apple:  two
Invalid price, please enter a number.
Enter the price for apple:  2
Enter the product name (or type 'done' to finish):  orange
Enter the price for orange:  4
Enter the product name (or type 'done' to finish):  biscuit
Enter the price for biscuit:  1
Enter the product name (or type 'done' to finish):  done
Shopping Cart Summary:
Item:  bread , Quantity:  1 , Total Price:  3.0
Item:  orange , Quantity:  2 , Total Price:  7.0
Item:  apple , Quantity:  1 , Total Price:  2.0
Item:  biscuit , Quantity:  1 , Total Price:  1.0
Total Cost:  13.0
```

```python
# Initialize an empty dictionary to store the shopping cart
shopping_cart = {}

while True:
    # Ask for the product name
    product_name = input("Enter the product name (or type 'done' to finish): ").lower()

    # Stop if the user types 'done'
    if product_name == 'done':
        break

    # Ask for the product price
    while True:
        try:
            product_price = float(input(f"Enter the price for {product_name}: "))
            break
        except ValueError:
            print("Invalid price, please enter a number.")

    # If the product is already in the cart, update the quantity and total price
    if product_name in shopping_cart:
        shopping_cart[product_name]['quantity'] += 1
        shopping_cart[product_name]['total_price'] += product_price
    # If it's a new product, add it to the cart
    else:
        shopping_cart[product_name] = {'quantity': 1, 'total_price': product_price}

# Display the summary of the cart
print("Shopping Cart Summary:")
total_cost = 0

for product, details in shopping_cart.items():
    quantity = details['quantity']
    total_price = details['total_price']
    print("Item: ", product, ", Quantity: ", quantity, ", Total Price: ",total_price)
    total_cost += total_price

# Show the overall total price
print("Total Cost: ", total cost)
```

Exercise 5

Given the following dataset of students and their grades represented as a list of dictionaries, you are required to complete the following tasks:

1) Extract and print the names of all students.
2) Identify and print the subjects being taught.
3) Calculate and print the average grade for each student.
4) Calculate and print the average grade for each subject.

```
students_grades = [
    {"name": "Alice", "subject": "Math", "grade": 85},
    {"name": "Bob", "subject": "Math", "grade": 90},
    {"name": "Alice", "subject": "English", "grade": 78},
    {"name": "Bob", "subject": "Science", "grade": 88},
    {"name": "Alice", "subject": "Science", "grade": 92},
    {"name": "Charlie", "subject": "Math", "grade": 80},
    {"name": "Charlie", "subject": "English", "grade": 85},
]
```

Exercise 5

```python
# 1. Extract names of students
student_names = set()
for record in students_grades:
    student_names.add(record['name'])

print("Student Names:", student_names)
```

```
Student Names: {'Alice', 'Charlie', 'Bob'}
```

```python
# 2. Identify subjects being taught
subjects = set()
for record in students_grades:
    subjects.add(record['subject'])

print("Subjects Being Taught:", subjects)
```

```
Subjects Being Taught: {'English', 'Math', 'Science'}
```

Exercise 5

```python
# 3. Calculate average grade for each student
student_dict = {}
for record in students_grades:
    name = record['name']
    grade = record['grade']
    if name not in student_dict:
        student_dict[name] = {'grades': [], 'average': 0}
    student_dict[name]['grades'].append(grade)

print(student_dict)

# Calculate average grades for students
for student, info in student_dict.items():
    avg_grade = sum(info['grades']) / len(info['grades'])
    student_dict[student]['average'] = avg_grade

print(student_dict)

print("\nAverage Grades per Student:")
for student, info in student_dict.items():
    print("Student:", student, ", Average: ", info['average'])
```

```
{'Alice': {'grades': [85, 78, 92], 'average': 0}, 'Bob': {'grades': [90, 88], 'average': 0}, 'Charlie': {'grades': [80, 85], 'average': 0}}
{'Alice': {'grades': [85, 78, 92], 'average': 85.0}, 'Bob': {'grades': [90, 88], 'average': 89.0}, 'Charlie': {'grades': [80, 85], 'average': 82.5}}

Average Grades per Student:
Student: Alice , Average:  85.0
Student: Bob , Average:  89.0
Student: Charlie , Average:  82.5
```

Exercise 5

```python
# 4. Calculate average grade for each subject
subject_dict = {}
for record in students_grades:
    subject = record['subject']
    grade = record['grade']
    if subject not in subject_dict:
        subject_dict[subject] = {'grades': [], 'average': 0}
    subject_dict[subject]['grades'].append(grade)

print(subject_dict)

# Calculate average grades for subjects
for subject, info in subject_dict.items():
    avg_grade = sum(info['grades']) / len(info['grades'])
    subject_dict[subject]['average'] = avg_grade

print(subject_dict)

print("\nAverage Grades per Subject:")
for subject, info in subject_dict.items():
    print("Subject: ",subject , ", Average Grade: ", info['average'])
```

```
{'Math': {'grades': [85, 90, 80], 'average': 0}, 'English': {'grades': [78, 85], 'average': 0}, 'Science': {'grades': [88, 92], 'average': 0}}
{'Math': {'grades': [85, 90, 80], 'average': 85.0}, 'English': {'grades': [78, 85], 'average': 81.5}, 'Science': {'grades': [88, 92], 'average': 90.0}}

Average Grades per Subject:
Subject:  Math , Average Grade:  85.0
Subject:  English , Average Grade:  81.5
Subject:  Science , Average Grade:  90.0
```

Exercise 6

Movie Collection Organizer

You are required to build a simple movie collection organizer in Python.

The program should:

- Allow the user to input the title, type (genre), and release year of movies to be added to the collection. The user can add multiple movies.
- If a movie with the same title already exists, update its information rather than adding it again.
- After the user has finished entering movies, display:
  - A list of all unique types in the collection (use a set).
  - A count of how many movies are in each genre.
  - A list of movies released before the year 2000.

Exercise 6

- Allow the user to input the title, genre, and release year of movies to be added to the collection. The user can add multiple movies.
- If a movie with the same title already exists, update its information rather than adding it again.

```python
#Exercise 6

# Movie Collection Organizer

movie_collection = {}

while True:
    title = input("Enter movie title (or 'done' to finish): ").strip()
    if title.lower() == 'done':
        break

    genre = input("Enter genre: ").strip()
    year = int(input("Enter release year: "))

    # If the movie already exists, update its info
    if title in movie_collection:
        print(f"Updating information for '{title}'.")
    else:
        print("adding a new movie")
    movie_collection[title] = (genre, year)
```

Exercise 6

- After the user has finished entering movies, display:
  - A list of all unique genres in the collection (use a set).

```
# Find all unique genres using a set
genres = set()

for info in movie_collection.values():
    genre = info[0]
    genres.add(genre)

print("\nUnique Genres:")
print(genres)
```

Exercise 6

- After the user has finished entering movies, display:
  - A count of how many movies are in each genre.

```
# Count how many movies per genre using a dictionary
genre_count = {}
for info in movie_collection.values():
    genre = info[0]
    if genre in genre_count:
        genre_count[genre] += 1
    else:
        genre_count[genre] = 1

print("\nMovies per Genre:")
for genre in genre_count:
    print(genre , ":", genre_count[genre])
```

Exercise 6

- After the user has finished entering movies, display:
  - A list of movies released before the year 2000.

```
# List of movies released before the year 2000
old_movies = []
for title, info in movie_collection.items():
    year = info[1]
    if year < 2000:
        old_movies.append(title)

print("\nMovies released before 2000:")
if len(old_movies) > 0:
    for movie in old_movies:
        print("-", movie)
else:
    print("None found.")
```